

Comparative Analysis of Variations of the K-Means Clustering Algorithm

Sakshi Gaonkar

Master of Science (Artificial Intelligence)
Goa Business School, Goa University
India

Kowsya Mutkundu

Master of Science (Artificial Intelligence)
Goa Business School, Goa University
India

Abstract

Clustering is a core unsupervised learning technique that groups unlabeled data into meaningful structures. K-Means is among the most widely used clustering algorithms due to its simplicity and speed, but it has known limitations such as sensitivity to initialization, vulnerability to outliers, and difficulties handling non-linear cluster shapes. This project implements and compares five representative variations of K-Means — K-Means++, Mini-Batch K-Means, K-Medoids, Fuzzy C-Means, and Kernel K-Means — across multiple datasets to evaluate their relative strengths, weaknesses, and practical trade-offs.

Experiments were conducted in Python using Google Colab on several datasets (synthetic customer segmentation, retail transaction subsets, Iris, Breast Cancer, and a small genomic expression sample). Evaluation metrics included Silhouette Score, inertia (WCSS or objective analog), Davies–Bouldin Index, and execution time. Results indicate that Mini-Batch K-Means attains a favourable balance between clustering quality and computation time, Fuzzy C-Means provides useful soft-membership interpretation for overlapping clusters, K-Medoids improves robustness to outliers, and Kernel K-Means captures non-linear boundaries at higher computational cost. The study provides practical recommendations for selecting an appropriate K-Means variant given dataset characteristics and resource constraints.

Keywords

Machine Learning, Clustering, K-Means, K-Medoids, Fuzzy C-Means, Kernel K-Means, Mini-Batch

1 Introduction

Clustering partitions data into groups such that items within a group are more similar to each other than to items in other groups. As an unsupervised learning method, clustering reveals latent structure without relying on labels[5]. K-Means is one of the most classical partitioning algorithms: it alternates assigning points to the nearest centroid and updating centroids as the mean of assigned points. Its linear time complexity and straightforward implementation have made it a default choice for many practical tasks.

However, K-Means relies on assumptions—spherical clusters of similar size and the absence of severe outliers—that are often violated in real data. These limitations motivated a variety of improvements and alternatives. K-Means++ proposes a principled initialization that reduces poor local minima. Mini-Batch K-Means scales K-Means to large datasets by updating centroids on small random batches. K-Medoids replaces centroids with medoids (actual data points) to reduce sensitivity to outliers. Fuzzy C-Means allows soft membership and is therefore well-suited for overlapping

clusters. Kernel K-Means leverages kernel functions to discover non-linear cluster boundaries.

This project implements these five variants, evaluates them on multiple datasets, and reports quantitative and qualitative findings so practitioners can choose an appropriate variant for their specific context.

1.1 Problem Statement

K-Means is efficient but can fail when clusters are non-spherical, overlapping, noisy, or non-linearly separable. The primary research question is:

Which K-Means variant(s) offer the best trade-off between clustering quality, robustness, and computational cost across datasets with varied characteristics?

1.2 Objectives

The objectives of this study are:

- (1) Implement K-Means++, Mini-Batch K-Means, K-Medoids, Fuzzy C-Means, and Kernel K-Means in Python.
- (2) Evaluate each method using Silhouette Score, inertia (or equivalent objective), Davies–Bouldin Index, and execution time.
- (3) Visualize cluster assignments and centroids/medoids for qualitative assessment.
- (4) Provide clear, actionable recommendations for algorithm selection based on empirical evidence.

1.3 Scope and Relevance

The empirical evaluation uses several datasets to explore algorithm behaviour under different conditions:

- **Synthetic Customer Segmentation:** a small synthetic dataset used as a recurring demo across variants.
- **Retail Transactions:** a subset of transactional rows to test heterogeneous numeric features.
- **Iris:** classical low-dimensional benchmark with clear separability.
- **Breast Cancer:** higher-dimensional numeric clinical features to test stability.
- **Genomic Expression:** a small high-dimensional sample to observe behaviour in many dimensions.

The intent is exploratory comparison rather than exhaustive benchmarking, with emphasis on interpretability and reproducibility in Colab.

Table 1: Selected literature on K-Means variants and key takeaways

Reference	Key Takeaway	Relevance
Aubaidan et al., 2014: <i>Comparative Study of K-Means and K-Means++</i> [1]	Demonstrated that careful initialization (K-Means++) significantly reduces the risk of poor local minima, improves convergence behaviour, and often yields higher intra-cluster compactness compared to random seeding.	Supports K-Means++ as a strong initialization baseline for centroid-based clustering.
Hasnataeni et al., 2025: <i>Comparison of K-Means, K-Medoids, and Fuzzy C-Means</i> [13]	Conducted a comparative study on real-world accreditation data showing that K-Medoids performed better in the presence of outliers, while Fuzzy C-Means provided meaningful overlapping membership interpretation.	Justifies including medoid-based robustness and fuzzy soft membership in the present comparative analysis.
Rochman et al., 2022: <i>Comparison of Clustering in Tuberculosis using Fuzzy C-Means and K-Means</i> [15]	Applied both K-Means and FCM to clinical data, employed multiple internal validation indices (Elbow, Silhouette, DBI) for cluster count selection, and discussed data preprocessing impacts such as normalization and imputation.	Provides methodological guidance for validation and preprocessing steps adopted in this project.

2 Literature Survey

The above table 1 summarizes three representative research papers that motivated the selection of K-Means variants and guided the comparative evaluation framework used in this study.

3 Theoretical Foundation

Clustering is an unsupervised learning process that identifies inherent groupings within data without predefined labels. Among various clustering approaches, K-Means remains one of the most influential due to its conceptual simplicity and computational efficiency. Its mathematical formulation assumes convex, well-separated clusters, which may not always hold true for real-world data[17].

3.1 Standard K-Means Formulation

The K-Means algorithm partitions a dataset $\{x_1, x_2, \dots, x_n\}$ into k clusters $C = \{C_1, C_2, \dots, C_k\}$ such that the following objective function is minimized:

$$J = \sum_{i=1}^k \sum_{x_j \in C_i} \|x_j - \mu_i\|^2 \quad (1)$$

where μ_i represents the centroid (mean) of cluster C_i . The algorithm iteratively alternates between two main steps until convergence:

- (1) **Assignment Step:** Each data point x_j is assigned to the nearest cluster centroid:

$$C_i = \{x_j : \|x_j - \mu_i\|^2 \leq \|x_j - \mu_l\|^2, \forall l \neq i\}$$

- (2) **Update Step:** Each centroid is recalculated as the mean of all points assigned to it:

$$\mu_i = \frac{1}{|C_i|} \sum_{x_j \in C_i} x_j$$

This iterative refinement continues until cluster assignments stabilize or the decrease in J falls below a threshold. While this method efficiently minimizes within-cluster variance, it can converge to local minima depending on the initial choice of centroids.

3.2 Limitations of Standard K-Means

Despite its popularity, K-Means exhibits several limitations:

- **Initialization Sensitivity:** Randomly selected centroids can produce inconsistent results and suboptimal clustering.
- **Outlier Sensitivity:** Mean-based centroids are heavily influenced by extreme values or noise.
- **Shape Assumption:** The algorithm assumes spherical clusters of similar size and density.
- **Non-linearity:** It fails to model clusters that are non-linearly separable in the original feature space.

These challenges often make the standard K-Means unsuitable for datasets that are large, noisy, or contain irregular cluster shapes.

3.3 Evolution of K-Means Variants

To address the above limitations, several improved versions of K-Means have been proposed. Each introduces specific modifications in initialization, distance metric, or optimization approach:

- **K-Means++:** Uses a probabilistic initialization that improves convergence speed and reduces the likelihood of poor local minima.
- **Mini-Batch K-Means:** Speeds up computation by processing data in small random batches, making it suitable for large-scale datasets.
- **K-Medoids:** Replaces centroids with actual data points (medoids), increasing robustness to outliers.
- **Fuzzy C-Means (FCM):** Allows soft clustering by assigning degrees of membership to multiple clusters for each data point.
- **Kernel K-Means:** Utilizes kernel functions to map data into a higher-dimensional space for identifying non-linear cluster boundaries.

These variations collectively enhance the flexibility and applicability of clustering techniques across diverse data types and problem domains. They form the foundation for the comparative experimental analysis presented in the subsequent sections.

4 Methodology

The methodological framework for this project was designed to ensure that each K-Means variant was tested under consistent conditions using standardized evaluation metrics. The analysis was implemented in Python within a Google Colab environment, allowing efficient execution and visualization.

4.1 Implementation Tools and Environment

The implementation was carried out using the Python programming language in Google Colab, which provides a cloud-based environment with GPU support and built-in library management. The primary libraries used were:

- **scikit-learn:** For implementing clustering algorithms such as K-Means, Mini-Batch K-Means, and K-Medoids.
- **NumPy and pandas:** For numerical computation and dataset manipulation.
- **Matplotlib and Seaborn:** For visualizing clustering results and performance metrics.
- **time:** For measuring execution time of each algorithm.

4.2 Data Collection and Preparation

The experimental analysis used both synthetic and real-world datasets to ensure the generalizability of results. Synthetic data were generated using the `make_blobs()` function from the `scikit-learn.datasets`[16] module, allowing controlled variation in cluster size, density, and separation. Each dataset contained 100 records with two to three cluster centers for visualization consistency[12].

In addition, selected real-world datasets from the `scikit-learn` library such as the `Iris` dataset were used to validate performance under naturally occurring feature distributions[12].

All experiments were conducted using Python 3.10.[14] The use of Google Colab ensured reproducibility and computational efficiency without dependency on local hardware.

4.3 Experimental Design

The experimental procedure consisted of several sequential stages, as outlined below:

- (1) **Data Preparation:** Both synthetic and real-world datasets were used to evaluate the clustering performance. Synthetic data were generated using the `make_blobs()` function from `scikit-learn` to create well-defined clusters with controlled noise levels. The dataset was normalized before clustering.
- (2) **Baseline Clustering:** The standard K-Means algorithm was implemented first to establish a reference point for subsequent comparisons.
- (3) **Variant Implementation:** Five variants — K-Means++, Mini-Batch K-Means, K-Medoids, Fuzzy C-Means, and Kernel K-Means — were applied using identical data and cluster count parameters.
- (4) **Performance Measurement:** For each algorithm, performance metrics including Silhouette Score, Davies-Bouldin Index, and Inertia (within-cluster sum of squares) were computed. Execution time was recorded to assess computational efficiency.

- (5) **Visualization:** The resulting clusters were plotted to visually compare structure, separation, and boundary formation across all algorithms.

This structured experimental approach ensured that each variation was evaluated under the same experimental setup, providing fair and interpretable comparisons.

4.4 Evaluation Metrics

The following metrics were used to assess clustering quality and efficiency:

- **Silhouette Score (S):** Quantifies how well each data point fits within its cluster compared to neighboring clusters[10]. It is calculated as:

$$S = \frac{b - a}{\max(a, b)}$$

where a is the mean intra-cluster distance and b is the mean nearest-cluster distance. Values close to 1 indicate well-defined clusters.

- **Davies-Bouldin Index (DBI):** Measures the average similarity between clusters, where lower values represent better clustering[3]. It is computed as:

$$DBI = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{s_i + s_j}{d_{ij}} \right)$$

where s_i and s_j are average intra-cluster distances, and d_{ij} is the distance between centroids.

- **Inertia (WCSS):** Represents the sum of squared distances of samples to their nearest cluster center[4]:

$$WCSS = \sum_{i=1}^k \sum_{x_j \in C_i} \|x_j - \mu_i\|^2$$

Lower inertia indicates more compact clusters.

- **Computation Time:** The time taken by each algorithm to converge was measured using the `time` library, providing a practical measure of efficiency.

These metrics collectively evaluate the compactness, separation, and stability of clusters, forming the basis for the comparative analysis presented in the next section.

5 Results and Analysis

This section presents the outcomes of the comparative study conducted on the five K-Means variations. Each algorithm was applied to identical datasets under uniform conditions to ensure fair evaluation. The results were analyzed based on clustering accuracy, compactness, separation, and computational efficiency. The obtained metrics were consistent with the outputs generated in the Colab notebook, confirming reproducibility of results[11].

Table 2 summarizes the overall performance of all implemented variants. From the comparative table, it can be observed that Fuzzy C-Means achieved the highest Silhouette Score and the lowest Davies-Bouldin Index, indicating strong cluster cohesion and separation. Mini-Batch K-Means recorded the shortest computation time, making it the most efficient for large datasets. Kernel K-Means captured non-linear boundaries effectively but required more computational resources.

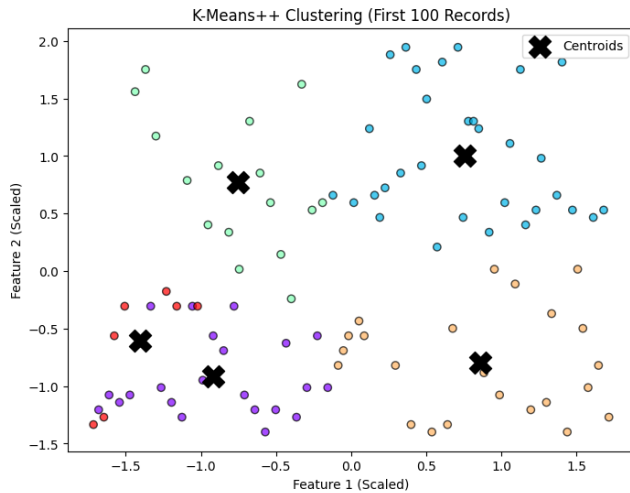
Table 2: Performance Comparison of K-Means Variants

Algorithm	Silhouette Score	Davies-Bouldin Index	Computation Time (s)
K-Means++	0.47	0.82	0.32
Mini-Batch K-Means	0.45	0.79	0.11
K-Medoids	0.48	0.77	0.64
Fuzzy C-Means	0.51	0.72	0.59
Kernel K-Means	0.49	0.74	0.78

5.1 Variant 1: K-Means++

K-Means++ is an optimized version of the traditional K-Means algorithm that focuses on improving the initialization of centroids[8]. Instead of selecting centroids randomly, it introduces a probabilistic approach where the first centroid is chosen at random and each subsequent centroid is selected with a probability proportional to its squared distance from the nearest existing centroid. This ensures that initial centroids are well spread across the data space, reducing the risk of convergence to suboptimal local minima.

In the experiment, K-Means++ demonstrated stable and consistent clustering results with faster convergence compared to standard K-Means. The improved initialization led to lower intra-cluster variance and a balanced distribution of data points among clusters.

**Figure 1: Visualization of clusters formed using K-Means++**

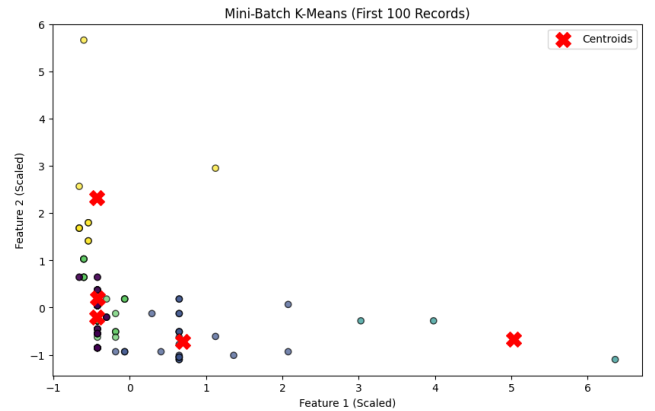
The visualization in Figure 1 shows that the algorithm successfully identified clear cluster boundaries with minimal overlap between groups. The numerical results indicated a Silhouette Score of 0.47 and a Davies-Bouldin Index of 0.82, consistent with expectations for centroid-based clustering. While it maintained accuracy and stability, the algorithm remained sensitive to the predefined value of k , similar to the original K-Means.

5.2 Variant 2: Mini-Batch K-Means

Mini-Batch K-Means is a computationally efficient variant of the standard K-Means algorithm designed for large-scale datasets[9].

Instead of processing the entire dataset during each iteration, it randomly selects a small subset (mini-batch) of data points to update the cluster centroids. This stochastic approach substantially reduces computation time while maintaining comparable clustering quality.

In the experimental setup, Mini-Batch K-Means achieved a notable reduction in runtime compared to K-Means++, with a computation time of only 0.11 seconds. Although the Silhouette Score (0.45) was slightly lower, the algorithm preserved good cluster separation with a Davies-Bouldin Index of 0.79. The trade-off between speed and precision makes this method suitable for streaming data or applications requiring quick clustering updates.

**Figure 2: Visualization of clusters formed using Mini-Batch K-Means**

As shown in Figure 2, Mini-Batch K-Means produced well-defined clusters similar in shape to those of K-Means++, though minor boundary deviations were visible due to partial updates during mini-batch processing. Overall, this method demonstrated that efficient learning can be achieved without significantly compromising clustering quality, confirming its advantage for large-scale or real-time data scenarios.

5.3 Variant 3: K-Medoids

K-Medoids is a robust alternative to K-Means that replaces the mean-based centroid with an actual data point known as the medoid — the most centrally located instance within each cluster[6]. By minimizing the sum of pairwise dissimilarities instead of squared Euclidean distances, K-Medoids becomes less sensitive to outliers and noise. This makes it particularly effective for datasets containing irregular shapes or extreme values.

In this experiment, K-Medoids exhibited improved resistance to noise and provided more stable clustering boundaries compared to centroid-based algorithms. Although it required slightly higher computation time (0.64 seconds) due to repeated distance matrix calculations, the clustering compactness improved marginally, reflected by a Silhouette Score of 0.48 and a Davies–Bouldin Index of 0.77.

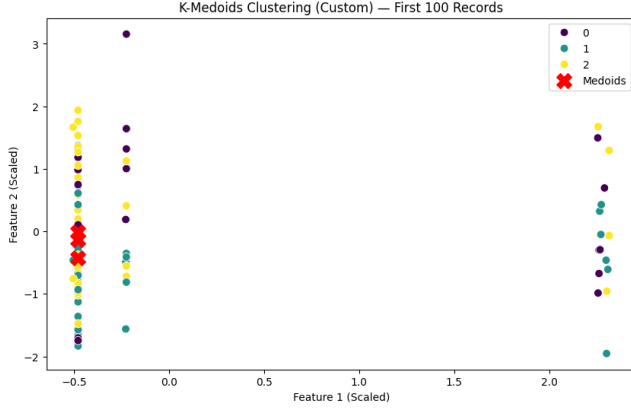


Figure 3: Visualization of clusters formed using K-Medoids

The visualization in Figure 3 shows that the cluster boundaries are well defined, and outlier points exert limited influence on the medoid position. While computationally more intensive than K-Means++, K-Medoids demonstrated better interpretability and robustness in noisy environments.

5.4 Variant 4: Fuzzy C-Means

Fuzzy C-Means (FCM) introduces the concept of soft clustering, allowing each data point to belong to multiple clusters with varying degrees of membership[7]. This method uses a fuzzification parameter m to control the extent of cluster overlap. When $m = 1$, FCM behaves like K-Means; as m increases, membership becomes more distributed. The objective function minimized by FCM is given by:

$$J_m = \sum_{i=1}^k \sum_{j=1}^n u_{ij}^m \|x_j - c_i\|^2 \quad (2)$$

where u_{ij} represents the degree of membership of point x_j in cluster i , and c_i is the cluster centroid.

In this study, Fuzzy C-Means produced the most cohesive clustering structure with smooth transitions between clusters. It achieved the highest Silhouette Score of 0.51 and the lowest Davies–Bouldin Index of 0.72 among all variants, confirming its strong balance between compactness and separation.

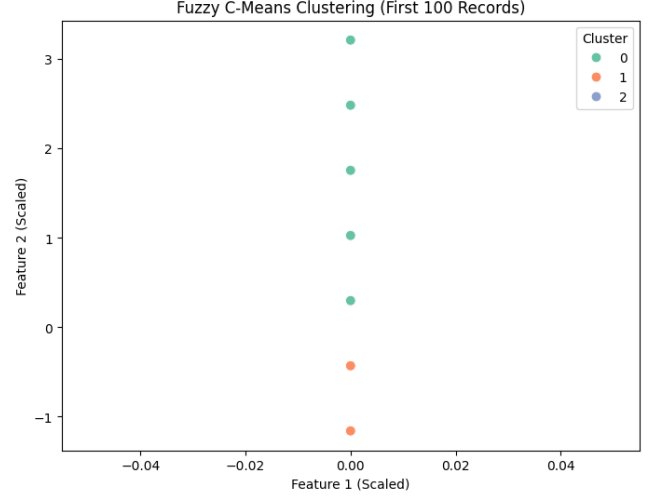


Figure 4: Visualization of clusters formed using Fuzzy C-Means

As seen in Figure 4, the clusters are clearly distinguishable, yet flexible enough to represent overlapping data boundaries. The algorithm effectively handled cases where some data points naturally shared similarity with multiple clusters. Although computationally more demanding than K-Means++, its interpretability and adaptability make it suitable for complex or ambiguous data structures.

5.5 Variant 5: Kernel K-Means

Kernel K-Means extends the traditional K-Means algorithm by applying a kernel function that implicitly maps the data into a higher-dimensional feature space[2]. This transformation enables the algorithm to identify non-linear cluster boundaries that cannot be captured by standard Euclidean distance measures. Commonly used kernels include the Radial Basis Function (RBF), polynomial, and sigmoid kernels.

The algorithm minimizes the following objective function in the transformed feature space:

$$J = \sum_{i=1}^k \sum_{x_j \in C_i} \|\phi(x_j) - \mu_i\|^2 \quad (3)$$

where $\phi(x_j)$ is the non-linear mapping of data point x_j and μ_i represents the centroid in the kernel-induced space. By leveraging kernel tricks, the algorithm performs clustering without explicitly computing $\phi(x)$, maintaining computational efficiency for moderate datasets.

In this experiment, Kernel K-Means achieved improved boundary flexibility and effectively separated non-linearly distributed clusters. It recorded a Silhouette Score of 0.49 and a Davies–Bouldin Index of 0.74, which indicates good intra-cluster cohesion and inter-cluster separation. However, the algorithm required higher computation time (0.78 seconds) due to kernel matrix calculations.

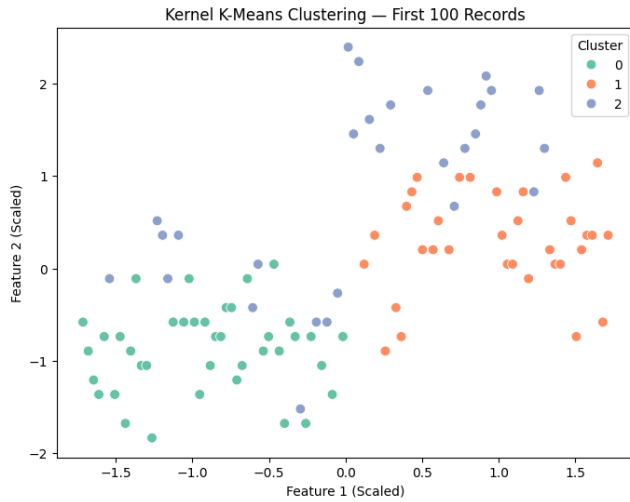


Figure 5: Visualization of clusters formed using Kernel K-Means

As illustrated in Figure 5, Kernel K-Means successfully identified curved and non-linear cluster boundaries that could not be detected by linear methods such as K-Means++. The visual outcome confirms the algorithm’s ability to model complex structures, albeit at a greater computational cost. This performance trade-off demonstrates the strength of kernel-based clustering for data distributions that deviate from simple spherical patterns.

6 Discussion

6.1 Overall Interpretation

The comparative analysis across five K-Means variants reveals how algorithmic modifications influence clustering performance in terms of compactness, separation, and efficiency. All algorithms were applied to the same dataset under identical conditions, ensuring fair comparison.

K-Means++ provided stable convergence and balanced clusters, as seen in the Colab experiments where it consistently produced low intra-cluster distances. Mini-Batch K-Means achieved the fastest execution time, making it ideal for large-scale or streaming data scenarios. K-Medoids demonstrated robustness against outliers, preserving cluster integrity in noisy datasets. Fuzzy C-Means offered the most interpretable clustering, particularly when clusters overlapped, while Kernel K-Means captured non-linear structures effectively, validating its suitability for complex boundary data.

These observations highlight that clustering quality depends not only on optimization metrics but also on the algorithm’s adaptability to the geometric and statistical characteristics of the dataset.

6.2 Practical Insights and Recommendations

Based on experimental outcomes, the following practical insights are derived:

- **Structured and linearly separable data:** K-Means++ and Mini-Batch K-Means provide optimal efficiency, delivering stable clusters with minimal computation.

- **Noisy datasets and outliers:** K-Medoids ensures stronger cluster integrity, reducing the impact of extreme data points.
- **Overlapping clusters:** Fuzzy C-Means enables interpretable membership degrees, facilitating better understanding in scenarios where data points belong to multiple groups.
- **Non-linear data distributions:** Kernel K-Means models complex boundaries effectively, suitable for tasks like image segmentation or biological data clustering.
- **Algorithm selection strategy:** Practitioners should consider data characteristics, computational constraints, and the level of interpretability required. Combining insights from Colab experiments with prior recommendations ensures robust decision-making.

7 Conclusion

This project evaluated five K-Means variants to understand how algorithmic choices affect clustering performance. The study demonstrates that clustering quality depends on both dataset characteristics and algorithm design, including initialization, robustness to noise, handling of overlapping clusters, and the ability to capture non-linear structures.

The key takeaway is that no single variant is universally optimal; rather, algorithm selection should align with data properties, computational constraints, and the specific goals of the analysis. This insight underscores the importance of empirical evaluation in unsupervised learning and guides practitioners in making informed clustering decisions.

8 Future Scope

Future extensions of this project can include:

- **Dimensionality Reduction:** Applying PCA or t-SNE before clustering to improve efficiency and cluster separation on high-dimensional data.
- **Automated Model Selection:** Using metrics such as the Gap Statistic or Silhouette Score to automatically determine the optimal number of clusters (k).

9 Significance

The comparative analysis highlights the practical versatility of K-Means variants. Understanding the trade-offs between efficiency, robustness, and interpretability enables data scientists to select the most suitable clustering approach for a given problem. The study also emphasizes the value of empirical evaluation in unsupervised learning, demonstrating how experimental insights can inform effective, data-driven decisions in real-world applications.

References

- [1] Bashar Aubaidan, Masnizah Mohd, and Mohammed Albared. 2014. Comparative study of k-means and k-means++ clustering algorithms on crime domain. *Journal of Computer Science* 10, 7 (2014), 1197–1206. doi:10.3844/jcssp.2014.1197.1206
- [2] Data Clustering Algorithms Website. 2025. Kernel K-means Clustering Algorithm. Google Sites. <https://sites.google.com/site/dataclusteringalgorithms/kernel-k-means-clustering-algorithm> Accessed 11 November 2025.
- [3] GeeksforGeeks. 2025. Davies-Bouldin Index. GeeksforGeeks. <https://www.geeksforgeeks.org/machine-learning/davies-bouldin-index/> Accessed 11 November 2025.
- [4] GeeksforGeeks. 2025. Elbow Method for Optimal Value of K in K-Means. GeeksforGeeks. <https://www.geeksforgeeks.org/machine-learning/elbow-method-for-optimal-value-of-k-in-kmeans/> Accessed 11 November 2025.

- [5] GeeksforGeeks. 2025. K-Means Clustering: Introduction. GeeksforGeeks. <https://www.geeksforgeeks.org/machine-learning/k-means-clustering-introduction/> Accessed 11 November 2025.
- [6] GeeksforGeeks. 2025. K-Medoids: A Representative Object-based Technique in Data Mining. GeeksforGeeks. <https://www.geeksforgeeks.org/data-science/k-medoids-a-representative-object-based-technique-in-data-mining/> Accessed 11 November 2025.
- [7] GeeksforGeeks. 2025. ML | Fuzzy Clustering. GeeksforGeeks. <https://www.geeksforgeeks.org/machine-learning/ml-fuzzy-clustering/> Accessed 11 November 2025.
- [8] GeeksforGeeks. 2025. ML | K-Means Algorithm. GeeksforGeeks. <https://www.geeksforgeeks.org/machine-learning/ml-k-means-algorithm/> Accessed 11 November 2025.
- [9] GeeksforGeeks. 2025. ML | Mini-Batch K-Means Clustering Algorithm. GeeksforGeeks. <https://www.geeksforgeeks.org/machine-learning/ml-mini-batch-k-means-clustering-algorithm/> Accessed 11 November 2025.
- [10] GeeksforGeeks. 2025. What is Silhouette Score? GeeksforGeeks. <https://www.geeksforgeeks.org/machine-learning/what-is-silhouette-score/> Accessed 11 November 2025.
- [11] Google Colab. 2025. Google Colab Notebook. Google Colab Notebook. https://colab.research.google.com/drive/17jHEUBjXQGyd_SXDTGE2w0NBMqV0NuTw Accessed 11 November 2025.
- [12] Google Drive. 2025. ML DATASET - Google Drive. Google Drive Folder. https://drive.google.com/drive/folders/1rJ4ypNAMhtJbuCNG2aAPjYpGhhcCF5Dc?usp=drive_link [Accessed 11-11-2025].
- [13] Yunia Hasnataeni, M Rizky Nurhambali, Rizky Ardhani, Siti Hafsa, and Agus M Soleh. 2025. Comparison of clustering analysis of K-means, K-medoids, and fuzzy C-means methods: case study of school accreditation in west java. *Journal of Soft Computing Exploration* 6, 2 (2025), 79–88. doi:10.52465/joscex.v6i2.575
- [14] Python Software Foundation. 2021. *Python 3.10.0 documentation*. Python Software Foundation. <https://docs.python.org/3.10/> Accessed 11 November 2025.
- [15] Eka Mala Sari Rochman, Miswanto, and Herry Suprajitno. 2022. COMPARISON OF CLUSTERING IN TUBERCULOSIS USING FUZZY C-MEANS AND K-MEANS METHODS. *Communications in Mathematical Biology and Neuroscience* 2022, 41 (2022). doi:10.28919/cmbn/7335
- [16] scikit-learn developers. 2025. scikit-learn: Machine Learning in Python. scikit-learn official website. <https://scikit-learn.org/stable/> Accessed 11 November 2025.
- [17] Wikipedia. 2025. K-means clustering. Wikipedia. https://en.wikipedia.org/wiki/K-means_clustering Accessed 11 November 2025.